

HospiTrack Production Deployment Checklist

Use this checklist before deploying to production and as part of ongoing operations.

Pre-Deployment Checks

Code Quality

- [] All tests pass: `pytest tests/ -v`
- [] Code linted with Flake8: `flake8 . --config=.flake8`
- [] Code formatted with Black: `black . --check`
- [] No security vulnerabilities in dependencies: `pip-audit` or `safety check`
- [] All TODO/FIXME comments reviewed and addressed
- [] Code reviewed by at least one other developer (if team)

Data Validation

- [] `data/us_er.parquet` file exists and is up to date
- [] Parquet file size is reasonable (<100 MB recommended)
- [] Data includes required columns: `name`, `address`, `latitude`, `longitude`, `detail_state`
- [] ML model files exist (if ML demo enabled):
- [] `models/triage_model.pkl`
- [] `models/triage_encoders.pkl`
- [] Sample queries return valid results (test with major cities)

Configuration Files

- [] `.env.example` is up to date with all variables
- [] Sensitive data is NOT in `.env.example` or committed to Git
- [] `Dockerfile.prod` builds successfully: `docker build -f Dockerfile.prod -t hospitrack-er:test .`
- [] `render.yaml` or platform config is correct (ports, paths, env vars)
- [] `.dockerignore` excludes unnecessary files (tests, docs, `.git`, etc.)

Documentation

- [] `README.md` is complete and accurate
- [] `DEPLOYMENT.md` deployment steps verified
- [] API documentation generates correctly: visit `/docs`
- [] All medical disclaimers are present and clear
- [] License information is included (if applicable)

Testing

- [] **Unit tests:** Core logic (geocoding, sorting, triage) has test coverage
- [] **Integration tests:** API endpoints return correct responses
- [] **Load testing:** Application handles expected traffic (use `ab`, `locust`, or `k6`)
- [] **Browser testing:** Frontend works on Chrome, Firefox, Safari, Edge

- [] **Mobile testing:** Responsive design works on iOS and Android devices
 - [] **Geolocation testing:** Browser location prompt works correctly
-

Security Checks

Application Security

- [] **No hardcoded secrets:** API keys, passwords in environment variables only
- [] **HTTPS enforced:** All production traffic uses TLS/SSL
- [] **CORS configured:** Only allowed origins can access API (if restricted)
- [] **Input validation:** All user inputs sanitized (location, severity, etc.)
- [] **Rate limiting:** Implement rate limits to prevent abuse (optional for MVP)
- [] **Error handling:** No sensitive information leaked in error messages
- [] **Health check endpoint:** `/healthz` accessible without authentication

Docker Security

- [] Running as non-root user (see `Dockerfile.prod`)
- [] Minimal base image used (`python:3.11-slim`)
- [] No unnecessary packages installed
- [] Security scanning passed: `docker scan hospitracker:production` (if available)
- [] Multi-stage build reduces image size and attack surface

Data Privacy

- [] No PII (Personally Identifiable Information) logged or stored
 - [] User addresses hashed before caching (implemented in `geolocation.py`)
 - [] No persistent user tracking (cookies, sessions)
 - [] Geocoding cache does not persist between deployments (in-memory only)
 - [] Compliance with HIPAA/GDPR requirements (if applicable)
-

Deployment Execution

Platform Setup (Render Example)

- [] Render account created and GitHub connected
- [] Repository access granted to Render
- [] Web service created with correct settings:
- [] Runtime: Docker
- [] Dockerfile path: `./Dockerfile.prod`
- [] Instance type: Starter (minimum) or higher
- [] Auto-deploy enabled for `main` branch

Environment Variables Configured

- [] `PORT=8000`
- [] `PYTHONUNBUFFERED=1`
- [] `HOSPITRACK_DATA_PATH=/app/data`

- [] `GEOCODING_CACHE_SIZE=1000` (or higher for production)
- [] `ML_DEMO_ENABLED=true` (or `false` if disabling ML)
- [] `LOG_LEVEL=INFO` (use `DEBUG` only for troubleshooting)
- [] `GUNICORN_WORKERS=4` (adjust based on instance CPU cores)
- [] Optional: `SENTRY_DSN` for error tracking
- [] Optional: `ALLOWED_HOSTS` for host validation

Build & Deploy

- [] Docker image builds successfully (check Render/platform logs)
- [] Build time is reasonable (<10 minutes)
- [] Image size is optimized (<500 MB recommended)
- [] Health checks pass during startup
- [] Application starts without errors in logs

Post-Deployment Verification

Functional Testing

- [] **Landing page loads:** Visit `https://your-app.com/`
- [] **Health check responds:** `curl https://your-app.com/healthz`
- [] **API docs load:** Visit `https://your-app.com/docs`
- [] **Search functionality:**
 - [] Test with address: "San Francisco, CA"
 - [] Test with ZIP code: "10001"
 - [] Test with browser geolocation (on HTTPS)
- [] **Map rendering:** Hospital markers appear correctly
- [] **Explore page:** State filter and sorting work
- [] **Triage demo:** Both rule-based and ML modes return results
- [] **Error handling:** Invalid inputs show user-friendly error messages

Performance Testing

- [] **Response times:**
 - [] Health check: <100ms
 - [] Search API: <2 seconds (first query), <500ms (cached)
 - [] Explore API: <1 second
 - [] Triage API: <1 second
- [] **Load testing:** Application handles 100 concurrent users (adjust based on plan)
- [] **Memory usage:** Container stays under allocated RAM
- [] **CPU usage:** Average <50% under normal load

Monitoring Setup

- [] **Application logs:** Accessible via platform dashboard or log aggregator
- [] **Error tracking:** Sentry or similar tool configured (optional)
- [] **Uptime monitoring:** UptimeRobot, Pingdom, or platform health checks
- [] **Performance monitoring:** New Relic, Datadog, or similar (optional)

- ☐ **Alerts configured:**
 - ☐ Downtime alert (health check fails)
 - ☐ High error rate alert (>5% errors)
 - ☐ Resource alert (CPU/memory >80%)
-

Operational Readiness

Documentation

- ☐ **Internal runbook:** Step-by-step troubleshooting guide
- ☐ **API documentation:** Public and up to date at `/docs`
- ☐ **User guide:** Instructions for end users (if needed)
- ☐ **Change log:** Document version history and breaking changes

Backup & Recovery

- ☐ **Data backup plan:** Regular backups of `data/us_er.parquet` (if updated)
- ☐ **Model backup:** ML model files backed up externally
- ☐ **Configuration backup:** `.env` templates and platform settings documented
- ☐ **Rollback plan:** Process to revert to previous deployment
- ☐ **Disaster recovery:** Steps to restore service if platform fails

Scaling Plan

- ☐ **Vertical scaling:** Plan to upgrade instance size if needed
 - ☐ **Horizontal scaling:** Configure auto-scaling (if platform supports)
 - ☐ **Database offloading:** Plan to move data to database if dataset grows
 - ☐ **CDN setup:** Serve static files via CDN for global users (optional)
 - ☐ **Caching strategy:** Redis/Memcached for high-traffic scenarios (optional)
-

Compliance & Legal

Medical Disclaimers

- ☐ Landing page includes “call 911 for emergencies” warning
- ☐ Home search page includes medical disclaimer
- ☐ Triage demo clearly labeled as “not for clinical use”
- ☐ ML demo explicitly states “trained on synthetic data”
- ☐ Footer includes “for informational purposes only” statement

Terms & Privacy

- ☐ Privacy policy created and linked (if collecting any data)
- ☐ Terms of service created and linked
- ☐ Cookie consent banner (if using cookies for analytics)
- ☐ GDPR compliance (if serving EU users)
- ☐ HIPAA compliance assessment (if handling PHI - not currently)

Licensing

- ☐ Code license specified (MIT, Apache, proprietary)
 - ☐ Third-party license compliance:
 - ☐ FastAPI: MIT License ☒
 - ☐ Folium: MIT License ☒
 - ☐ Scikit-learn: BSD License ☒
 - ☐ Leaflet.js: BSD License ☒
 - ☐ OpenStreetMap: ODbL License ☒ (attribution required)
 - ☐ Data license compliance:
 - ☐ CMS Hospital Compare: Public domain ☒
 - ☐ Attribution provided where required
-

Performance Optimization

Backend

- ☐ **Parquet file optimized:** Using columnar format for fast queries
- ☐ **Geocoding cached:** LRU cache enabled (1000+ entries)
- ☐ **Gunicorn workers:** Set to `(2 * CPU cores) + 1`
- ☐ **Database indexing:** Not applicable (using in-memory data)
- ☐ **Query optimization:** Pandas operations optimized (vectorized)
- ☐ **API response caching:** Consider adding for static queries (optional)

Frontend

- ☐ **Static assets minified:** CSS/JS compressed (optional)
- ☐ **Images optimized:** Use WebP or optimized PNGs (not applicable)
- ☐ **Lazy loading:** Map loads only when needed
- ☐ **Code splitting:** Separate JS bundles for each page (optional)
- ☐ **CDN for static files:** Use CDN for global performance (optional)

Infrastructure

- ☐ **Regional deployment:** Deploy to region closest to users
 - ☐ **Auto-scaling enabled:** Scale horizontally under load (if supported)
 - ☐ **Health checks tuned:** Appropriate intervals and timeouts
 - ☐ **Resource limits set:** Prevent runaway resource usage
-

Monitoring & Maintenance

Daily Checks (Automated)

- ☐ Health check endpoint responding
- ☐ Error rate <1%
- ☐ Response times within SLA
- ☐ No critical errors in logs

Weekly Checks (Manual)

- [] Review error logs for patterns
- [] Check resource usage trends
- [] Verify uptime metrics
- [] Review user feedback (if available)

Monthly Checks

- [] Update dependencies: `pip list --outdated`
- [] Security updates: `pip-audit`
- [] Review and rotate logs
- [] Update hospital dataset if new data available
- [] Review and optimize costs

Quarterly Checks

- [] Disaster recovery drill (test rollback)
- [] Load testing to verify performance
- [] Security audit
- [] Documentation review and update
- [] Evaluate new features based on user feedback

Troubleshooting Quick Reference

Application Won't Start

1. Check logs: `docker logs hospitracker` or platform logs
2. Verify data files exist: `docker exec hospitracker ls /app/data`
3. Check environment variables are set correctly
4. Verify Docker image built successfully

High Response Times

1. Check geocoding cache hit rate
2. Verify Gunicorn workers are sufficient
3. Check database/Parquet file size
4. Monitor CPU/memory usage
5. Enable request logging to identify slow endpoints

Health Check Fails

1. Verify `/healthz` endpoint exists
2. Check if app is listening on correct port
3. Review startup logs for errors
4. Increase health check timeout/start period
5. Test locally: `curl http://localhost:8000/healthz`

Map Not Rendering

1. Check browser console for JavaScript errors
2. Verify Leaflet.js CDN is accessible

3. Check if data includes valid lat/lon
4. Test API response format

ML Demo Errors

1. Verify model files exist: `models/triage_model.pkl`
 2. Check `ML_DEMO_ENABLED=true` in environment
 3. Review triage API logs for errors
 4. Test with simple inputs first
-

Success Criteria

Technical

- ☒ Uptime >99.5%
- ☒ Average response time <2 seconds
- ☒ Error rate <1%
- ☒ Zero security incidents
- ☒ All tests passing

Business

- ☒ Application accessible to target users
- ☒ Search results are accurate and relevant
- ☒ User feedback is positive
- ☒ No legal/compliance issues
- ☒ Operating costs within budget

User Experience

- ☒ Landing page loads in <2 seconds
 - ☒ Search returns results in <3 seconds
 - ☒ Map is interactive and responsive
 - ☒ Mobile experience is smooth
 - ☒ Error messages are clear and helpful
-

Post-Launch Tasks

Immediate (Week 1)

- ☐ Monitor logs daily for unexpected errors
- ☐ Collect user feedback via form or email
- ☐ Fix any critical bugs within 24 hours
- ☐ Announce launch to stakeholders

Short-term (Month 1)

- ☐ Analyze usage patterns (most searched locations, complaints)
- ☐ Optimize based on performance data
- ☐ Plan feature improvements

- ☐ Write blog post or case study (optional)

Long-term (Quarter 1)

- ☐ Implement user-requested features
- ☐ Expand dataset (more hospitals, real-time wait times)
- ☐ Explore monetization options (B2B API, white-label)
- ☐ Consider open-sourcing or publicizing project

Checklist Version: 1.0

Last Updated: December 2024

Recommended Review Frequency: Before each production deployment